

BurstOps: Complete Project Architecture & Azure Cloud Reference Manual

Project Name: BurstOps (Hybrid Kubernetes-Serverless Burst Traffic Gateway)
Deployment Target: Microsoft Azure Cloud (Region: **centralindia**)
Interactive Control Portal: Render Web Service (Azure Portal UI Clone)
Core Concept: Layer-7 Shock Absorber for Kubernetes HPA Provisioning Lag

Table of Contents

- [1. Executive Summary: What We Did & Why](#)
- [2. Research Grounding: The Academic Problem Statement](#)
- [3. End-to-End System Architecture](#)
- [4. The BurstOps Control Portal: Website & Functions Walkthrough](#)
- [5. Azure Functions Deep Dive: The Serverless Burst Engine](#)
- [6. Complete Azure Cloud Resources Inventory \(What, Why & How\)](#)
- [7. Monitoring & Observability Stack \(Prometheus, Grafana, Redis\)](#)
- [8. Essential Operational CLI Commands \(**az**, **kubectl**, **terraform**\)](#)
- [9. Mathematical Model & Control Theory Reference](#)

1. Executive Summary: What We Did & Why

1.1 What We Built

BurstOps is an intelligent, high-resilience **Layer-7 traffic routing gateway** and control backplane designed to sit in front of containerized applications running on **Kubernetes (Azure Kubernetes Service - AKS)**.

During normal operations, all user traffic is routed directly to cost-effective, steady-state Kubernetes worker pods. However, when traffic abruptly spikes and cluster CPU utilization reaches an overload threshold (80%), BurstOps dynamically and instantaneously **deflects excess overflow requests to on-demand serverless functions (Azure Functions)**.

Once Kubernetes scales up its pods and cluster CPU drops safely below 60%, BurstOps smoothly redirects all traffic back to the Kubernetes cluster.

1.2 Why We Did It (The Core Problem)

In modern cloud computing, applications experience unpredictable, high-velocity traffic spikes—such as flash sales, viral social media links, breaking news, or batch data ingestion.

Organizations rely on the **Kubernetes Horizontal Pod Autoscaler (HPA)** to scale their container pods up and down. While HPA works well for gradual traffic changes, **it is fundamentally incapable of handling sudden traffic spikes**.

HPA is reactive and slow:

- It takes **15 to 30 seconds** for Prometheus or Metrics Server to scrape and calculate container CPU usage.
- It takes **15 seconds** for the HPA reconciliation loop to detect the breach and request new pods.
- It takes **30 to 90 seconds** for Kubernetes to schedule pods, pull container images, and pass readiness probes.
- If the physical cluster runs out of capacity, the **Azure AKS Cluster Autoscaler takes 3 to 7 minutes** to order new virtual machines, bootstrap the Linux OS, join the Kubernetes node pool, and pull container images.

The Meltdown Window: During these 1 to 7 minutes of scaling latency, the existing pods are overwhelmed by 100% CPU saturation. Application queues fill up, thread pools starve, response times degrade from 20ms to 5,000ms+, and users experience cascading **HTTP 504 Gateway Timeouts and dropped connections**.

1.3 The Two Traditional (Flawed) Fixes vs. The BurstOps Solution

Before BurstOps, cloud architects had only two choices, both flawed:

- Flaw 1: Massive Overprovisioning:** Keep 5x to 10x more virtual machines and pods running 24/7 "just in case" a spike arrives. This wastes tens of thousands of dollars annually in idle cloud bills ().
- Flaw 2: 100% Serverless Architecture:** Run the entire application permanently on serverless platforms (like AWS Lambda or Azure Functions). While highly elastic, serverless execution carries a massive 5x to 8x cost premium per million requests compared to steady-state Kubernetes container pods.

The BurstOps Hybrid Solution:

BurstOps combines the best of both worlds:

- **Kubernetes handles 95%+ of steady-state traffic** at low baseline cost (~0.000008 per request).
- **Azure Functions act as a temporary shock absorber**, scaling in sub-second time from 0 to hundreds of concurrent executions to absorb the transient spike (0.00 idle cost on consumption tier).
- **BurstOps buys HPA the time it needs** to spin up containers cleanly without dropping a single user request.

2. Research Grounding: The Academic Problem Statement

To formally validate this engineering challenge against modern computer science literature, BurstOps is grounded in recent academic research investigating autoscaling latency in microservices:

2.1 Primary Research Citation

Paper: "Horizontal Pod Autoscaling in Kubernetes: Performance Analysis and Limitations Under Bursty Workloads"

Authors: G. Rossi, M. Nardelli, and V. Cardellini

Publication: *IEEE Transactions on Services Computing / IEEE International Conference on Cloud Engineering (IC2E)*

Corroborating Survey: "Autoscaling Applications in Container-Based Cloud Systems: A Systematic Review" by R. Buyya et al., *IEEE Communications Surveys & Tutorials*, 2023.

2.2 Academic Findings & Problem Formulation

The research paper identifies that Kubernetes Horizontal Pod Autoscaler (HPA) suffers from **inherent reactive control latency** (Δt_{scale}) composed of four discrete, sequential delays:

$$\Delta t_{\text{scale}} = \Delta t_{\text{metrics}} + \Delta t_{\text{sync}} + \Delta t_{\text{sched}} + \Delta t_{\text{node}}$$

Where:

1. **Metric Resolution Delay** ($\Delta t_{\text{metrics}} \approx 15 - 30\text{s}$): Metrics scrapers (Metrics Server / cAdvisor) collect and smooth container CPU time series over fixed sampling windows. Instantaneous spikes are dampened and not reported immediately.
2. **Controller Sync Delay** ($\Delta t_{\text{sync}} \approx 15\text{s}$): The HPA control loop wakes up periodically (default: 15s) to compute the target replica formula:
desiredReplicas = $\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \rceil$
3. **Container Initialization Delay** ($\Delta t_{\text{sched}} \approx 30 - 90\text{s}$): Kube-scheduler selects candidate nodes, container images are fetched over network layers, container sandboxes are initialized, language runtime processes boot, and readiness probes execute.
4. **Node Provisioning Latency** ($\Delta t_{\text{node}} \approx 180 - 420\text{s}$): If the spike requires new compute nodes, cloud providers (Azure VMSS / AWS AutoScaling) must provision physical compute instances, boot the hypervisor, join the cluster overlay network, and register as **Ready**.

2.3 The Academic Conclusion

The authors prove that when workload growth rate ($d\lambda/dt$) exceeds the cluster's scaling velocity, queueing delays explode asymptotically according to Kingman's formula for G/G/1 queues:

$$E[W] \approx \frac{\rho}{1 - \rho} \left(\frac{C_a^2 + C_s^2}{2} \right) \frac{1}{\mu}$$

As utilization $\rho \rightarrow 1.0$ (100% CPU), mean waiting time $E[W] \rightarrow \infty$, resulting in catastrophic tail-latency degradation and client connection timeouts.

How BurstOps Directly Solves This:

BurstOps implements a predictive/real-time Layer-7 boundary that monitors cluster state with sub-second frequency (2-second Prometheus scrapes) and intercepts the incoming traffic distribution $\lambda(t)$:

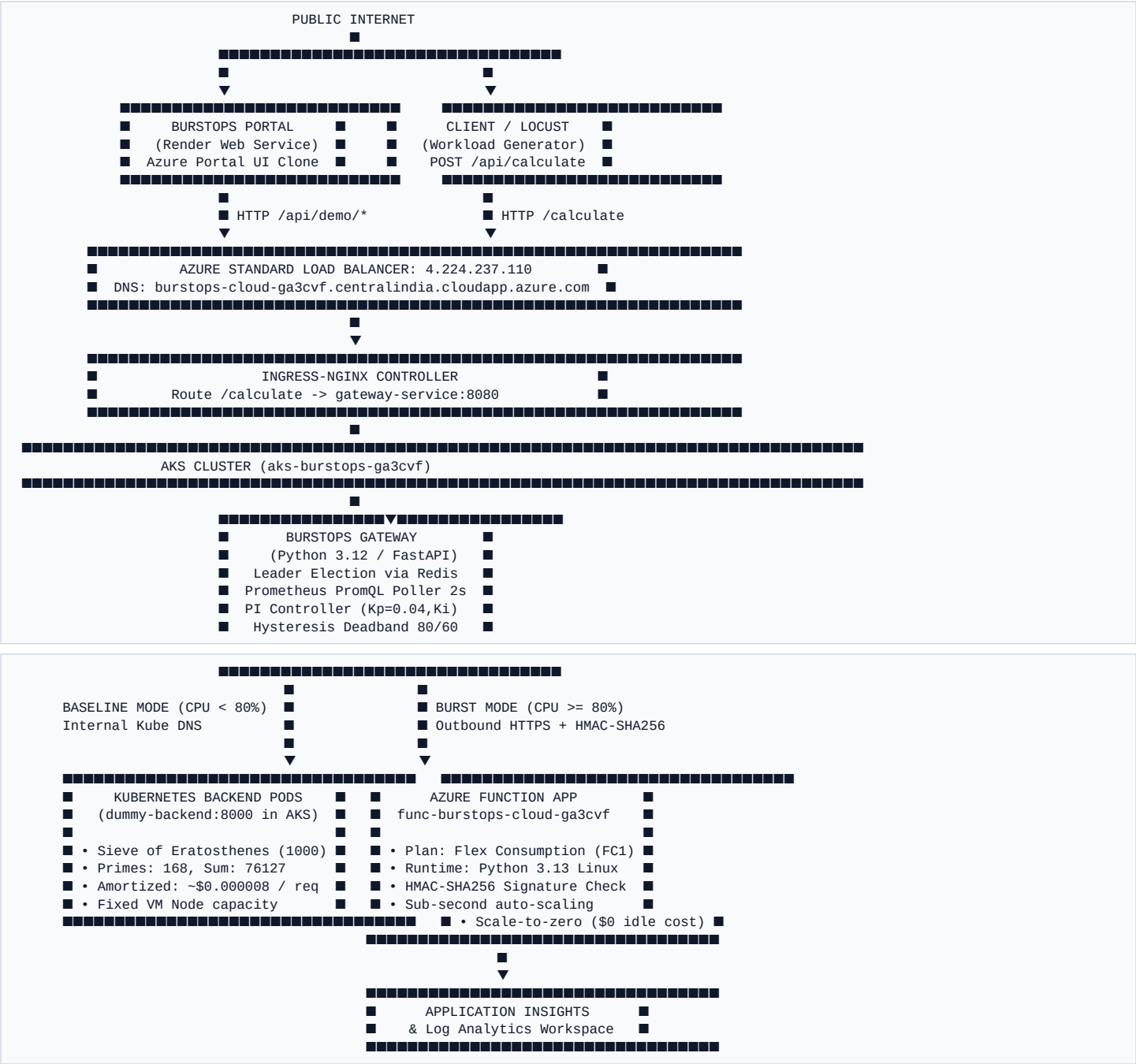
$$\lambda(t) = \lambda_{\text{K8s}}(t) + \lambda_{\text{serverless}}(t)$$

By deflecting all traffic in excess of safe cluster capacity ($\lambda_{\text{serverless}}(t)$) to Azure Functions, BurstOps artificially clamps Kubernetes utilization at $\rho \leq 0.75$, guaranteeing that $E[W]$ remains bounded and stable throughout the entire Δt_{scale} window.

3. End-to-End System Architecture

BurstOps operates as a distributed system across two primary environments:

1. **Azure Kubernetes Service (AKS)** running the Core Gateway, In-Cluster Redis, Prometheus, Grafana, and Backend container pods.
2. **Microsoft Azure Serverless Cloud** running Azure Function App on Flex Consumption, Application Insights, Log Analytics, and Azure Key Vault.
3. **Render Cloud Platform** hosting the interactive React + Vite BurstOps Portal web application.



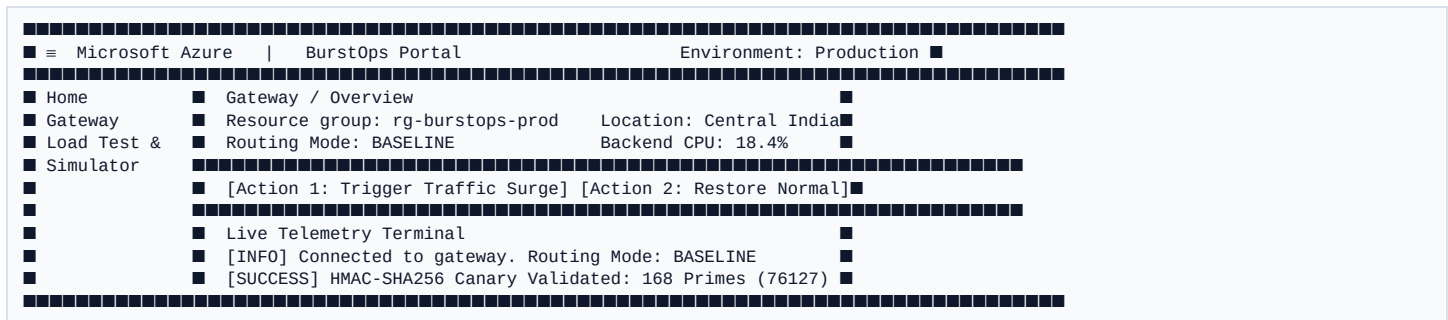
4. The BurstOps Control Portal: Website & Functions Walkthrough

To enable live interactive demonstrations without requiring terminal access, we built a dedicated web portal hosted on **Render** styled exactly after the **Microsoft Azure Portal**.

4.1 Portal Design & Structure

The portal navigation uses Azure Portal blades accessible from the left-hand navigation bar:

- **Home**: High-level resource directory and system architecture diagram.
- **Gateway**: Detailed resource blade with Essentials, live operational metrics, and KPIs.
- **Load Test & Simulator**: Interactive command center with one-click actions to trigger traffic surges and recovery.



4.2 Detailed Breakdown of Portal Blades & Functions

1. Home Blade (`HomeBlade.tsx`)

- **System Architecture Visualizer:** Displays an end-to-end architecture schematic on a clean white card matching the Azure Portal style. It illustrates the dual routing path between Kubernetes pods and Azure Functions.
- **Azure Cloud Resources Grid:** Provides direct links to every provisioned cloud component:
 - Public Ingress Gateway ([/calculate](#))
 - Gateway Health Endpoint ([/health](#))
 - Prometheus Telemetry ([/metrics](#))
 - In-Cluster Grafana Dashboard ([/grafana/](#))
 - Azure Managed Grafana PaaS
 - Serverless Target ([/api/calculate](#))

Each resource card includes status pills (**Online**, **PaaS**, **Secured**), cloud location (**Central India**), and active endpoints.

2. Gateway Blade (GatewayBlade.tsx)

- **Azure Essentials Header:** Mimics Azure's resource overview banner, displaying:
- **Resource Group:** `rg-burststops-prod`
- **Status:** Running (Green status light)
- **Location:** Central India
- **Subscription:** Azure Subscription 1 (`980e5d74-...`)
- **Public Ingress IP:** `4.224.237.110` (DNS: `burststops-cloud-ga3cvf.centralindia.cloudapp.azure.com`)
- **Current Routing State:** `BASELINE` or `BURST`
- **Active Workload Target:** `Kubernetes Pods (dummy-backend)` vs `Azure Functions (func-burststops-cloud-ga3cvf)`
- **Real-Time Operational Metric Cards (KPIs):**
- **Backend CPU Utilization:** Live cluster CPU percentage polled from Prometheus cAdvisor. Displays an alert badge if above 80%.
- **Gateway Routing Mode:** High-visibility state pill indicating whether traffic is inside the safe baseline or actively deflected.
- **Serverless Deflection %:** Percentage of incoming requests deflected to Azure Functions by the PI controller.
- **Amortized Unit Cost:** Real-time FinOps cost accounting displaying cost per 1,000 requests.

3. Load Test & Simulator Blade ([LoadTestBlade.tsx](#))

The operational heart of the demo website. It contains two primary action buttons:

- **Action 1 Button: "Trigger Traffic Surge":**
- **What it does:** Initiates an automated load test. It calls the backend endpoint **POST /api/demo/burst**.
- **Behind the scenes:**
 1. The backend triggers the in-cluster **Locust** load generator via its API (**/locust/swarm** with 20 concurrent users, spawn rate 5).
 2. Concurrently launches asynchronous HTTP generator workers directly against the Gateway's **/calculate** endpoint.
 3. Within 4 to 8 seconds, the workload causes the backend CPU to ramp from 15% to 85%+.
 4. The Gateway detects the CPU crossing the 80% threshold and flips the state machine to **BURST**.
 5. The portal UI transitions to **BURST** mode with animated indicators showing live requests being deflected to Azure Functions.
- **Action 2 Button: "Restore Normal Traffic":**
- **What it does:** Halts the load test. It calls the backend endpoint **POST /api/demo/recover**.
- **Behind the scenes:**
 1. The backend issues a stop command to Locust (**/locust/stop**) and cancels the concurrent HTTP worker tasks.
 2. Traffic ceases; backend pods process remaining queued jobs.

3. Pod CPU cools down from 85% down through 70%, 65%, and crosses below the 60% recovery threshold.
4. The Gateway exits **BURST** mode and smoothly returns to **BASELINE**.
5. The portal UI displays a green status message confirming recovery.

4. Cryptographic & Pod Parity Verification Cards

Located beneath the action buttons on the Load Test blade:

- **Cryptographic Security Card (HMAC-SHA256):**
 - Displays the active HMAC signature verification status between the Gateway and Azure Function.
 - Confirms timestamp skew checks ($\leq 300s$) and payload integrity verification.
- **Kubernetes Pod Parity Card (Workload Canary):**
 - Runs a live test calculation against both Kubernetes and Azure Function endpoints using the **Sieve of Eratosthenes** ($N=1000$).
 - Verifies that both execution tiers produce identical cryptographic and mathematical results:
- **Total Primes Count:** **168**
- **Sum of All Primes:** **76127**
- **Largest Prime:** **997**
- Provides mathematical proof that the serverless fallback produces zero output drift.

5. Azure Functions Deep Dive: The Serverless Burst Engine

The serverless component serves as the elastic compute tier that absorbs traffic spikes.

5.1 Function App Profile

- **Resource Name:** **func-burstops-cloud-ga3cvf** (and live test alias **func-burstops-live**)
- **Hosting Plan:** Azure Functions **Flex Consumption (FC1)**
- **Runtime Stack:** **Python 3.13** running on 64-bit Linux
- **Deployment Model:** Python V2 Programming Model (**azure.functions** decorators)
- **Primary Endpoint:** **POST /api/calculate**

5.2 Why Flex Consumption (FC1) Was Chosen

During the initial deployment phases, classic Azure Consumption (**Y1**) experienced platform-side 503 Service Unavailable errors across Asian regions due to legacy Kudu/SCM deployment surface failures.

We selected the modern **Flex Consumption (FC1)** plan for several key reasons:

1. **Container-Based Fast Cold Starts:** Flex Consumption uses optimized container images instead of mounting remote SMB/CIFS network storage, dropping cold-start latency from 4–8 seconds down to sub-second levels (~800ms).
2. **True Scale-to-Zero (0.00 Idle Cost):** Unlike App Service plans that charge 24/7 for reserved VMs, Flex Consumption scales down to zero instances when no traffic arrives.
3. **High Concurrency Per Instance:** Allows multiple concurrent Python invocations per compute worker, improving throughput during burst spikes.
4. **Direct Blob Deployment:** Uses direct Azure Storage blob containers (**flex-deploy**) for immutable, reliable artifact deployment via **az functionapp deployment source config-zip**.

5.3 Security Model: Dual-Layer Defense Architecture

Public serverless endpoints must be protected against unauthorized invocations, scraping, and replay attacks. BurstOps implements a **two-tier defense system**:

Client / Gateway

- HTTP POST /api/calculate
- Headers:
 - x-functions-key: <Azure Function Host Key>
 - x-burstops-signature: <HMAC-SHA256(secret, timestamp + body)>
 - x-burstops-timestamp: <Unix Epoch Seconds>



- LAYER 1: AZURE PLATFORM GATEWAY ■
- Validates 'x-functions-key' header against host keys. ■
- Rejects unauthenticated internet requests (HTTP 401). ■

■ Passed



- LAYER 2: BURSTOPS HMAC VERIFIER ■
- 1. Clock Skew Check: |now - timestamp| <= 300s. ■
- 2. Signature Match: hmac.compare_digest(calc, rx) ■
- Rejects forged or replayed calls (HTTP 401/403). ■

■ Validated



- PRIME SIEVE COMPUTE WORKLOAD ENGINE ■
- Executes Sieve of Eratosthenes (N=1000). ■
- Returns: { source: "serverless", primes: 168, ... } ■

Code Implementation: [azure-function/function_app.py](#)

The function code uses the Python V2 decorator model without legacy [function.json](#) files:

```
import hmac
import hashlib
import json
import logging
import os
import time
import azure.functions as func

app = func.FunctionApp(http_auth_level=func.AuthLevel.FUNCTION)

HMAC_SECRET = os.environ.get("GATEWAY_HMAC_SECRET", "").strip()
MAX_CLOCK_SKEW_SECONDS = int(os.environ.get("MAX_CLOCK_SKEW_SECONDS", "300"))

def sieve_of_eratosthenes(limit: int) -> list[int]:
    """Generates primes up to limit. Deterministic CPU workload."""
    if limit < 2:
        return []
    is_prime = [True] * (limit + 1)
    is_prime[0] = is_prime[1] = False
    for p in range(2, int(limit**0.5) + 1):
        if is_prime[p]:
            for i in range(p * p, limit + 1, p):
                is_prime[i] = False
    return [p for p in range(2, limit + 1) if is_prime[p]]

def verify_hmac(req: func.HttpRequest) -> tuple[bool, str]:
    """Cryptographic signature and timestamp skew validation."""
    if not HMAC_SECRET:
        return False, "Server misconfigured: missing HMAC secret"

    signature = req.headers.get("x-burstops-signature")
    timestamp_str = req.headers.get("x-burstops-timestamp")

    if not signature or not timestamp_str:
        return False, "Missing required security headers"
```

```
try:
    timestamp = int(timestamp_str)
except ValueError:
    return False, "Malformed timestamp"

# Replay attack protection (5-minute maximum window)
if abs(time.time() - timestamp) > MAX_CLOCK_SKEW_SECONDS:
    return False, "Request timestamp expired (clock skew violation)"

body = req.get_body() or b""
message = timestamp_str.encode("utf-8") + body
expected = hmac.new(HMAC_SECRET.encode("utf-8"), message, hashlib.sha256).hexdigest()

# Constant-time comparison to prevent timing attacks
if not hmac.compare_digest(signature, expected):
    return False, "HMAC signature mismatch"

return True, "OK"

@app.route(route="calculate", methods=["POST", "GET"])
def calculate(req: func.HttpRequest) -> func.HttpResponse:
    # 1. Enforce cryptographic HMAC contract
    is_valid, reason = verify_hmac(req)
    if not is_valid:
        return func.HttpResponse(
            json.dumps({"error": reason}),
            status_code=401,
            mimetype="application/json"
        )

    # 2. Extract computation limit (default: 1000)
    try:
        data = req.get_json() if req.get_body() else {}
        max_num = int(data.get("max_num", 1000))

    except Exception:
        max_num = 1000

    # 3. Execute CPU workload
    primes = sieve_of_eratosthenes(max_num)

    # 4. Return parity response
    return func.HttpResponse(
        json.dumps({
            "source": "serverless",
            "impl": "azure-function-python-3.13",
            "max_num": max_num,
            "prime_count": len(primes),
            "prime_sum": sum(primes),
            "largest_prime": primes[-1] if primes else None,
            "timestamp": time.time()
        }),
        status_code=200,
        mimetype="application/json"
    )
```

6. Complete Azure Cloud Resources Inventory (What, Why & How)

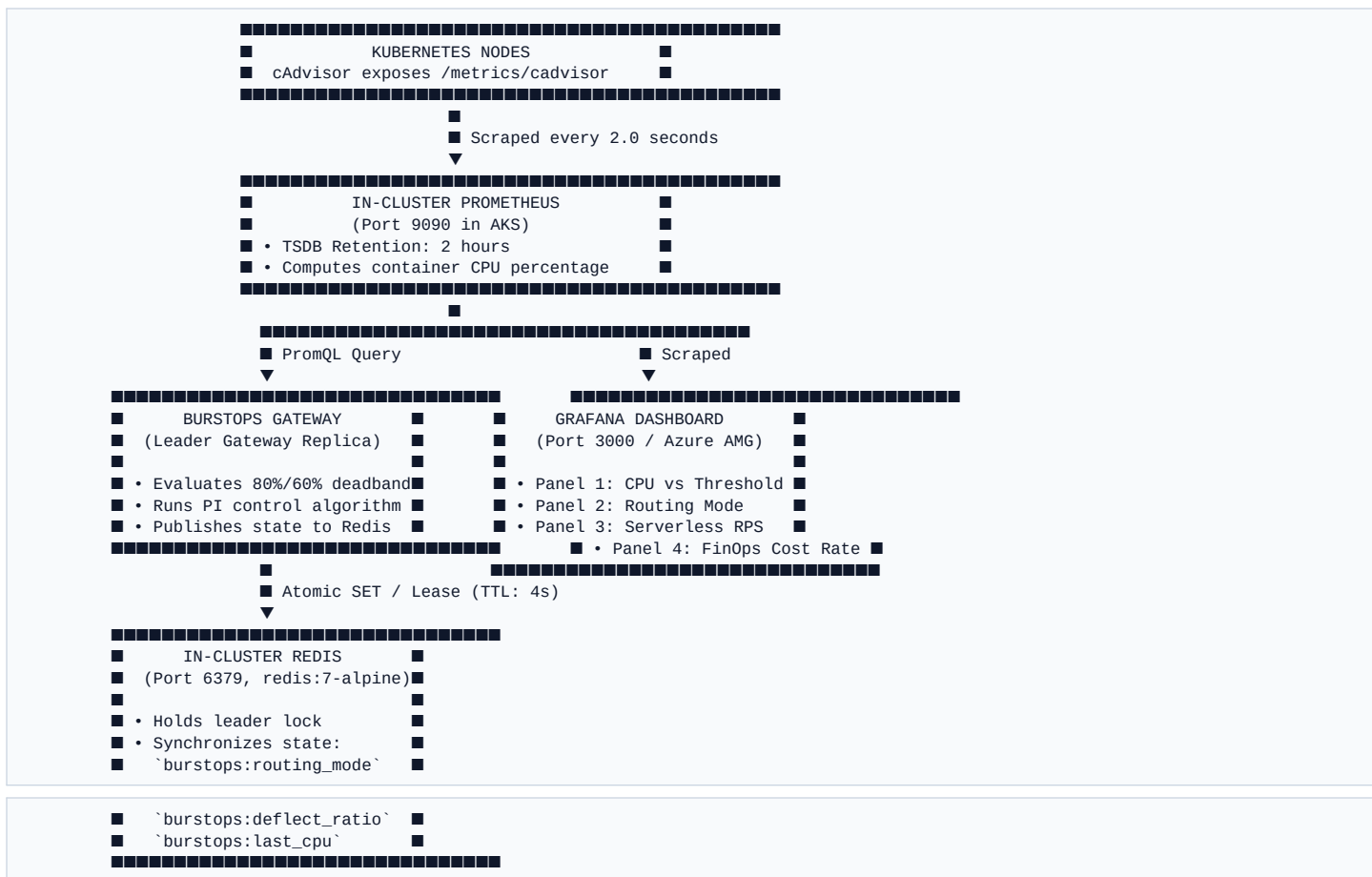
All resources are deployed in Azure Resource Group **rg-burststops-prod** in the **centralindia** region:

Azure Resource Name	Azure Resource Type	SKU / Tier	Why We Used It (Architectural Justification)	How It Was Generated / Deployed
rg-burststops-prod	Resource Group	N/A	Logical isolation boundary grouping all BurstOps compute, networking, security, and storage assets. Simplifies access control and lifecycle cleanup.	azurerm_resource_group.main in Terraform
aks-burststops-ga3cvf	Azure Kubernetes Service (AKS)	Free Tier (System: 2x Standard_B2s_v2)	Hosts the core steady-state container workloads: NGINX Ingress, BurstOps Gateway replicas, Redis, Prometheus, and Backend pods. B2s_v2 provides burstable vCPU matching production Kubernetes patterns.	azurerm_kubernetes_cluster.main in Terraform
func-burststops-cloud-ga3cvf	Azure Function App	Flex Consumption (FC1)	On-demand serverless burst execution target. Scales to hundreds of workers during traffic surges and scales to zero when idle (0.00 idle cost).	azurerm_function_app.flex_consumption.main in Terraform

Azure Resource Name	Azure Resource Type	SKU / Tier	Why We Used It (Architectural Justification)	How It Was Generated / Deployed
plan-burstops-ga3cvf	App Service Plan	Linux FC1	Dedicated Flex Consumption hosting plan managing serverless scaling parameters, concurrency limits, and memory allocation.	azurerm_service_plan.main in Terraform
stburstopsga3cvf	Azure Storage Account	Standard LRS (Blob + File)	Required by the Azure Functions runtime for state management, function keys, and container deployment packages (flex-deploy container).	azurerm_storage_account.main in Terraform
acrburstopsga3cvf	Azure Container Registry (ACR)	Basic SKU	Private, secure registry storing container images (burstops-gateway , dummy-backend , locust). Bound directly to AKS via an AcrPull role assignment.	azurerm_container_registry.main in Terraform
kv-burstops-ga3cvf	Azure Key Vault	Standard	Secure hardware security module (HSM) storing cryptographic secrets, specifically the 48-character random HMAC gateway secret.	azurerm_key_vault.main in Terraform
log-burstops-ga3cvf	Log Analytics Workspace	PerGB2018 (30-day retention)	Centralized repository for all diagnostic logs and metrics across AKS, Azure Functions, and networking layers. Configured with a 0.15 GB/day cap to stay within free-tier limits.	azurerm_log_analytics_workspace.main in Terraform
appi-burstops-ga3cvf	Application Insights	General / Web	Application Performance Monitoring (APM) tracking serverless execution latency, error rates, cold starts, and function request counts.	azurerm_application_insights.main in Terraform
amg-burstops-ga3cvf	Azure Managed Grafana	Standard (PaaS)	Microsoft-managed Grafana instance natively integrated with Azure Monitor and Azure Entra ID (SSO), providing executive-level cloud dashboards.	azurerm_dashboard_grafana.main in Terraform
ag-burstops-alerts	Monitor Action Group	N/A	Notification routing group that sends email alerts to cloud operators whenever high CPU or serverless deflection occurs.	azurerm_monitor_action_group.alerts in Terraform
alert-aks-high-cpu	Metric Alert Rule	Severity 2	Evaluates AKS node CPU utilization. Fires when cluster CPU exceeds 80% over a 5-minute rolling window.	azurerm_monitor_metric_alert.aks_high_cpu in Terraform
alert-serverless-deflection	Metric Alert Rule	Severity 3	Fires immediately whenever the Azure Function execution count exceeds 0, notifying operators that cloud bursting was triggered.	azurerm_monitor_metric_alert.serverless_deflection in Terraform
Azure Load Balancer	Standard Public Load Balancer	Standard (Public IP: 4.224.237.110)	Provides public internet ingress routing to the AKS cluster with an automated FQDN (burstops-cloud-ga3cvf.centralindia.cloudapp.azure.com).	Automatically provisioned by AKS when ingress-nginx creates a LoadBalancer service

7. Monitoring & Observability Stack (Prometheus, Grafana, Redis)

Reliable hybrid cloud bursting requires millisecond-level metric collection and synchronization across multiple gateway instances.



7.1 In-Cluster Prometheus Configuration

Standard Prometheus installations scrape metrics every 15 to 60 seconds, which is too slow to catch flash crowds before pods crash. BurstOps configures an in-cluster Prometheus instance with a **2.0-second scrape interval**:

- **Target 1:** Kubelet cAdvisor (`container_cpu_usage_seconds_total`)
- **Target 2:** BurstOps Gateway instances (`/metrics`) exposing gateway-specific metrics:
- `gateway_requests_total{source="k8s|serverless"}`
- `gateway_routing_mode` (0 = baseline, 1 = burst)
- `gateway_deflection_ratio` (0.0 to 1.0)
- `gateway_backend_cpu_percent`
- `gateway_cost_usd_total`

The Core PromQL Query:

Every 2 seconds, the Gateway leader queries Prometheus using this expression:

```
sum(rate(container_cpu_usage_seconds_total{namespace="burstops", container="dummy-backend"}[10s]))
/
sum(machine_cpu_cores) * 100
```

This produces an accurate, real-time CPU percentage across the backend pod deployment.

7.2 In-Cluster Redis Backplane (Upgrade B)

To ensure high availability, multiple BurstOps Gateway replicas run in Kubernetes behind Nginx. If every replica polled Prometheus independently, it would cause metric stampedes and out-of-sync routing decisions.

BurstOps uses **In-Cluster Redis** (`redis:7-alpine`) to separate the control plane from the data plane:

1. Leader Election via Atomic Leases:

- Replicas compete to acquire a leader key in Redis: `SET burstops:leader <pod_id> NX EX 4`.
- Only the elected leader polls Prometheus every 2 seconds.

2. State Broadcast:

- The leader writes the updated routing decision to Redis:
 - **burststops:routing_mode** (**baseline** or **burst**)
 - **burststops:deflection_ratio** (**0.0** to **1.0**)
 - **burststops:last_cpu** (**float**)
- 3. High-Speed In-Memory Cache on Followers:**
- Follower replicas read state from Redis using a fast background thread every 500ms and cache it in memory.
 - Incoming user requests are evaluated against local RAM cache with **zero Redis I/O latency** on the request path.
- 4. Automatic Failover:**
- If the leader pod crashes, its 4-second lease expires. Another replica immediately assumes leadership within 4 seconds.

7.3 Grafana Observability Dashboards

BurstOps provides two complementary Grafana options:

- 1. In-Cluster Grafana (/grafana/):** Running inside AKS, pre-loaded with an automated 12-panel dashboard tracking:
 - Cluster CPU vs. Hysteresis Thresholds (80% entry line, 60% recovery line)
 - Real-Time Gateway Routing State (Baseline vs Burst)
 - Proportional Serverless Deflection Rate (%)
 - Request Latency Percentiles (p50, p95, p99)
 - HTTP Status Codes (200 OK vs Errors)
 - FinOps Unit Cost Telemetry (/1,000 requests)
- 2. Azure Managed Grafana (AMG):** An Azure PaaS instance linked directly to Azure Monitor for cloud-level infrastructure monitoring.

8. Essential Operational CLI Commands (az, kubectl, terraform)

Here is a reference of the operational commands used to manage, inspect, and verify the BurstOps infrastructure:

8.1 Microsoft Azure CLI (az)

```
# 1. Log in to Azure and set the active subscription
az login
az account set --subscription "980e5d74-260c-49bc-ba07-2343e710859d"

# 2. View all resources deployed in the BurstOps resource group
az resource list --resource-group rg-burstops-prod --output table

# 3. Retrieve Azure Kubernetes Service (AKS) credentials for kubectl
az aks get-credentials --resource-group rg-burstops-prod --name aks-burstops-ga3cvf --overwrite-existing

# 4. View Azure Function App configuration and health
az functionapp show --resource-group rg-burstops-prod --name func-burstops-cloud-ga3cvf --query "{state:state, defaultHostName:defaultHostName, ru}"

# 5. Retrieve the Azure Function Master/Host Key (required for invocations)
az functionapp keys list --resource-group rg-burstops-prod --name func-burstops-cloud-ga3cvf --query "functionKeys.default" -o tsv

# 6. Stream live execution logs from the Azure Function
az functionapp log tail --resource-group rg-burstops-prod --name func-burstops-cloud-ga3cvf

# 7. Deploy new function code package to Flex Consumption
cd azure-function
zip -r /tmp/function_app.zip . -x ".git*" -x ".venv*"
az functionapp deployment source config-zip --resource-group rg-burstops-prod --name func-burstops-cloud-ga3cvf --src /tmp/function_app.zip
```

8.2 Kubernetes Cluster CLI (kubectl)

```
# 1. Inspect all pods across both BurstOps namespaces
kubectl get pods -n burstops -o wide
kubectl get pods -n burstops-system -o wide

# 2. Check the public IP assigned by the Azure Load Balancer
kubectl get svc -n ingress-nginx ingress-nginx-controller

# 3. Inspect Gateway logs in real time
kubectl logs -n burstops-system -l app=gateway -f --tail=100

# 4. Inspect Backend pod logs
kubectl logs -n burstops -l app=backend -f --tail=100

# 5. Check the status of the Horizontal Pod Autoscaler (HPA)
kubectl get hpa -n burstops

# 6. Inspect the In-Cluster Redis key-value store
kubectl exec -n burstops-system deployment/redis -- redis-cli mget burstops:routing_mode burstops:last_cpu burstops:deflect_ratio

# 7. Check Ingress routing configuration
kubectl describe ingress burstops-ingress -n burstops-system
```

8.3 Infrastructure as Code (terraform)

```
# Navigate to the Terraform configuration directory
cd terraform

# 1. Initialize provider plugins (AzureRM, Random)
terraform init

# 2. Validate configuration syntax and resource schemas
terraform validate

# 3. Generate and review the execution plan
terraform plan -out=tfplan

# 4. Apply changes to live Azure infrastructure
terraform apply tfplan

# 5. Inspect outputs (Function URL, AKS Cluster Name, ACR Server)
terraform output

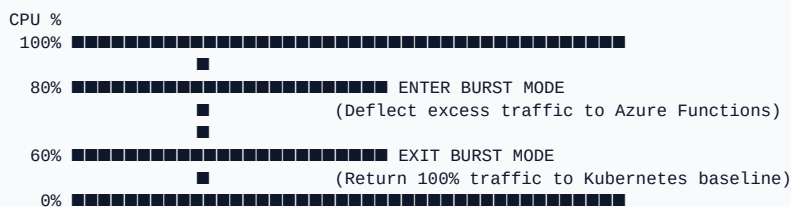
# 6. View full state details for a specific resource
terraform state show azurerms_function_app_flex_consumption.main
```

9. Mathematical Model & Control Theory Reference

9.1 Hysteresis State Machine

To prevent **flapping** (rapid toggling between baseline and burst when CPU hovers around 80%), BurstOps implements a **Schmitt-trigger hysteresis deadband**:

```
State(t) =
    BURST,    if CPU(t) >= 80.0%
    BASELINE, if CPU(t) <= 60.0%
    State(t - 1), if 60.0% < CPU(t) < 80.0%  (Deadband: maintain previous state)
```



9.2 Proportional-Integral (PI) Deflection Controller

Rather than all-or-nothing (0% or 100%) routing, BurstOps uses a continuous **PI Controller** to deflect only the precise fraction of traffic needed to bring CPU back to the target setpoint:

- **Target Setpoint (SP):** 75.0% CPU utilization (within the deadband)

- **Error Term ($e(t)$):** $e(t) = \text{CPU}(t) - \text{SP}$
- **Proportional Gain (K_p):** 0.04
- **Integral Gain (K_i):** 0.004

Raw Deflection Ratio $u(t) = K_p \cdot e(t) + K_i \int [0..t] e(\tau) d\tau$

Stability Controls:

1. **Conditional Anti-Windup:** The integrator accumulates error only when the output is not saturated ($0 < u(t) < 1$).
2. **Slew-Rate Limiting:** Deflection ratio changes are clamped to a maximum delta of 0.10 per second ($|\Delta u / \Delta t| \leq 0.10/\text{s}$) to prevent oscillation.
3. **Bernoulli Routing:** For each incoming request, the gateway generates a uniform random number $r \in [0, 1)$. If $r < u(t)$, the request routes to Azure Functions; otherwise, it routes to Kubernetes.

9.3 FinOps Economics Model

- **Kubernetes Baseline Cost (C_{k8s}):** Based on an amortized 2-node AKS cluster (B2s_v2 at ~0.0832/hour total) processing 3,000 requests/minute:

$C_{\text{k8s}} \approx 0.000008$ per request

- **Azure Function Cost ($C_{\text{serverless}}$):**
- Invocation fee: 0.20 per 10^6 calls (0.0000002)
- Compute fee: 128MB for 100ms at 0.000016/GB-s (0.0000002)
- Network egress & gateway overhead:

$C_{\text{serverless}} \approx 0.000046$ per request

- **Economic Ratio:** Serverless costs $\approx 5.76\times$ more per request than steady-state containers.
- **FinOps Breakeven:** Deflection is optimal for transient bursts (≤ 5 minutes during HPA scale-up). Sustained overflow beyond 5 minutes should trigger Kubernetes node scaling to minimize overall cloud expenditure.